



CSE357: COMBINATORIAL STUDIES



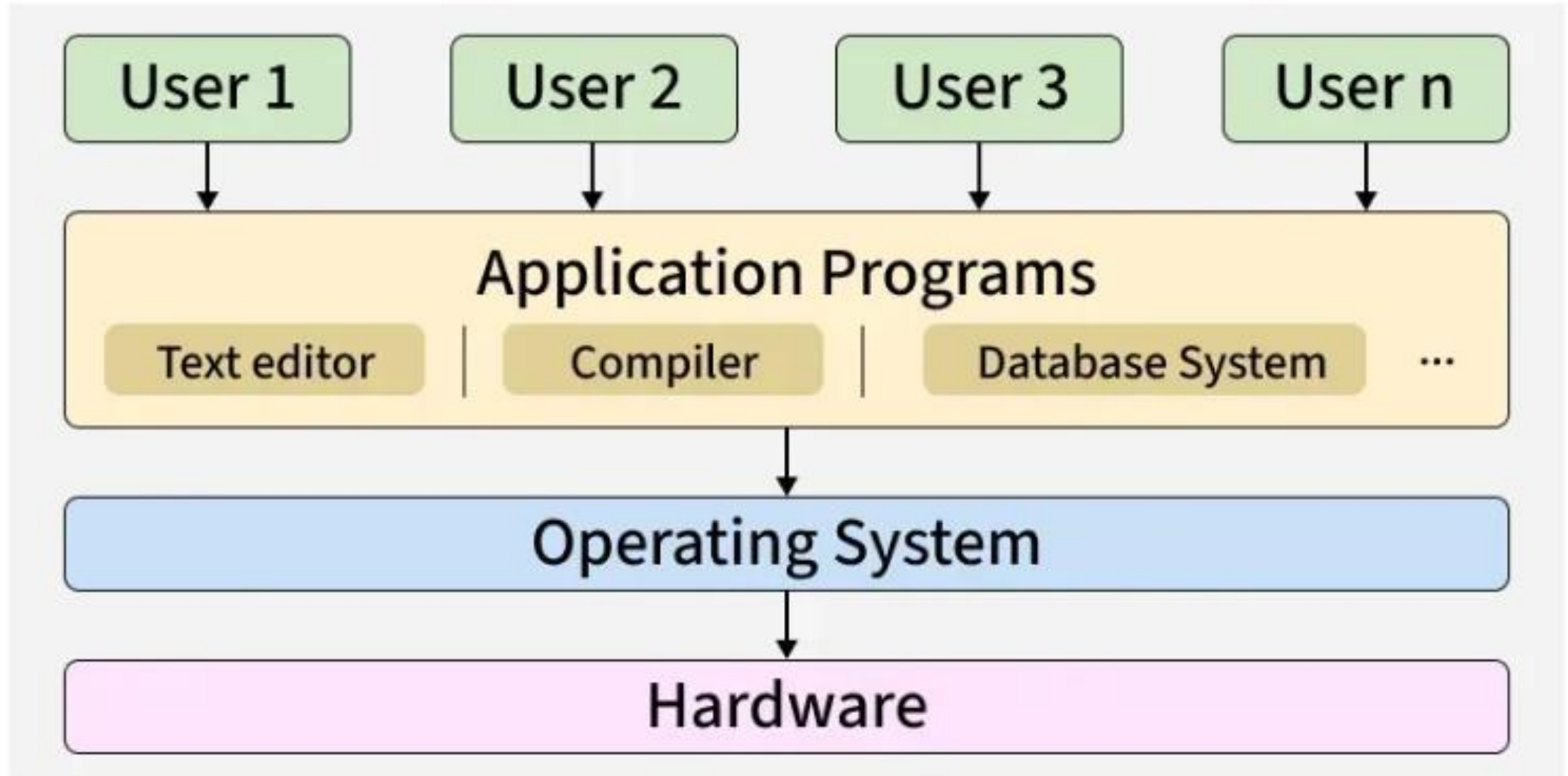
Unit I: Operating System Basics

- Foundations of Operating Systems
- Types of Operating Systems
- Memory Management
- Processor Scheduling Algorithms
- Process Synchronization
- Inter-processCommunication

Introduction to Operating System

- The software that manages computer hardware and provides services for computer programs.





Functions of an Operating System

1. Process Management

- Creation and termination of processes
- CPU scheduling
- Synchronization and communication

2. Memory Management

- Allocation and deallocation of memory
- Paging and segmentation
- Virtual memory management

3. File Management

- Create, read, write, delete files
- Directory management
- Access permissions

4. Device Management

- Controls I/O devices using device drivers
- Plug & Play

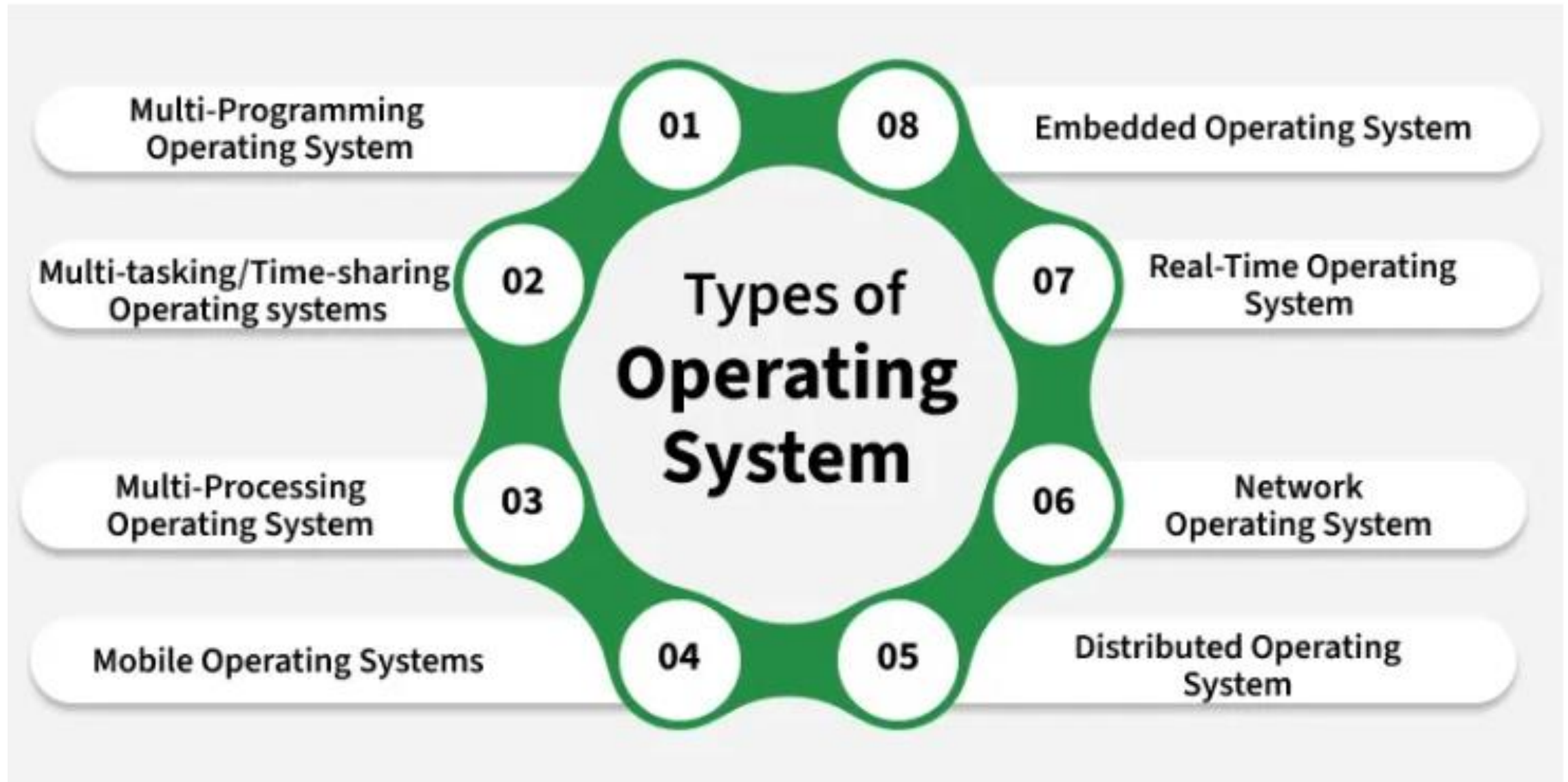
5. Security & Protection

- User authentication
- Access control (File permissions)

Types of Operating Systems:

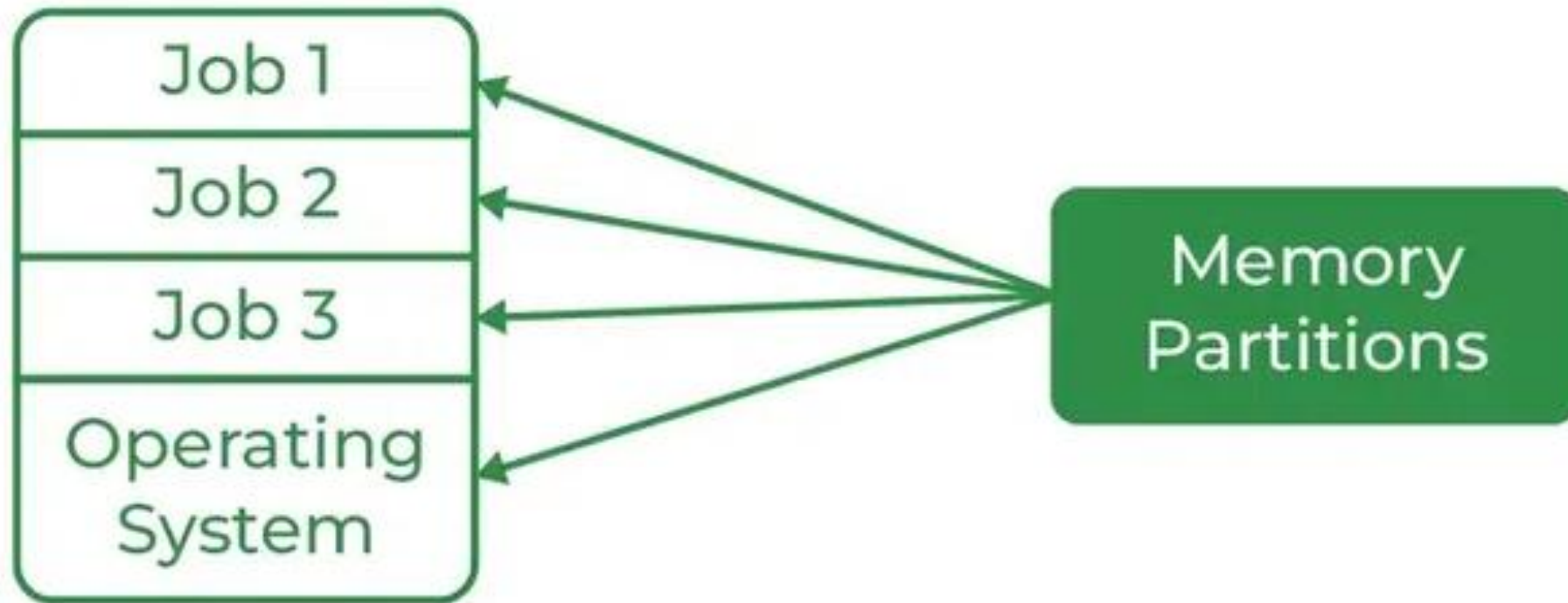
- Batch Operating System
- Time-Sharing Operating System
- Multiprogramming Operating System
- Multitasking Operating System
- Real-Time Operating System (RTOS)
- Distributed Operating System
- Network Operating System(NOS)
- Embedded Operating System
- Mobile Operating System
- Single-User Operating System
- Multi-User Operating System

Type	Key Feature	Examples of Operating Systems	Real-Life Examples
Batch OS	No user interaction	IBM OS/360, MVS	Payroll processing system, Bank statement generation
Time-Sharing OS	Multiple users	UNIX, Linux	University server accessed by many students, Airline reservation system
Multiprogramming OS	High CPU utilization	IBM OS/360, UNIX	Running compiler + printer + editor simultaneously
Multitasking OS	Multiple tasks	Windows, macOS	Listening to music while typing a document
RTOS	Time-critical	VxWorks, QNX	Air traffic control, Industrial robot control
Distributed OS	Multiple systems	Amoeba, LOCUS	Cloud computing systems, Distributed database servers
NOS	Network services	Windows Server, Novell NetWare	File sharing in offices, Network printers
Embedded OS	Dedicated devices	FreeRTOS, Embedded Linux	Washing machine, Microwave oven
Mobile OS	Smartphones	Android, iOS	Smartphones, Tablets
Single-User OS	One user	MS-DOS, Windows 95	Personal computer at home
Multi-User OS	Many users	UNIX, Linux	Banking servers, University mainframe



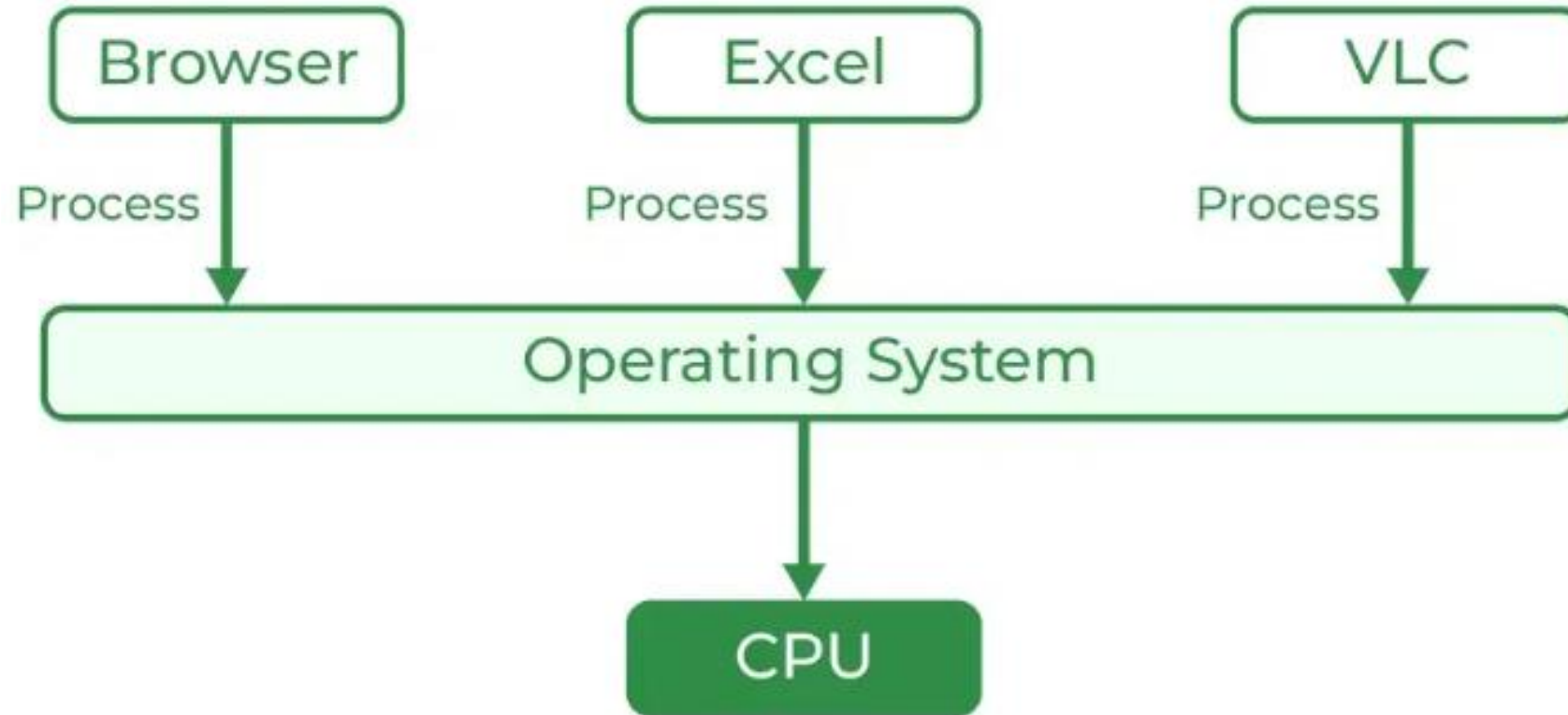
Types of Operating System

Multiprogramming



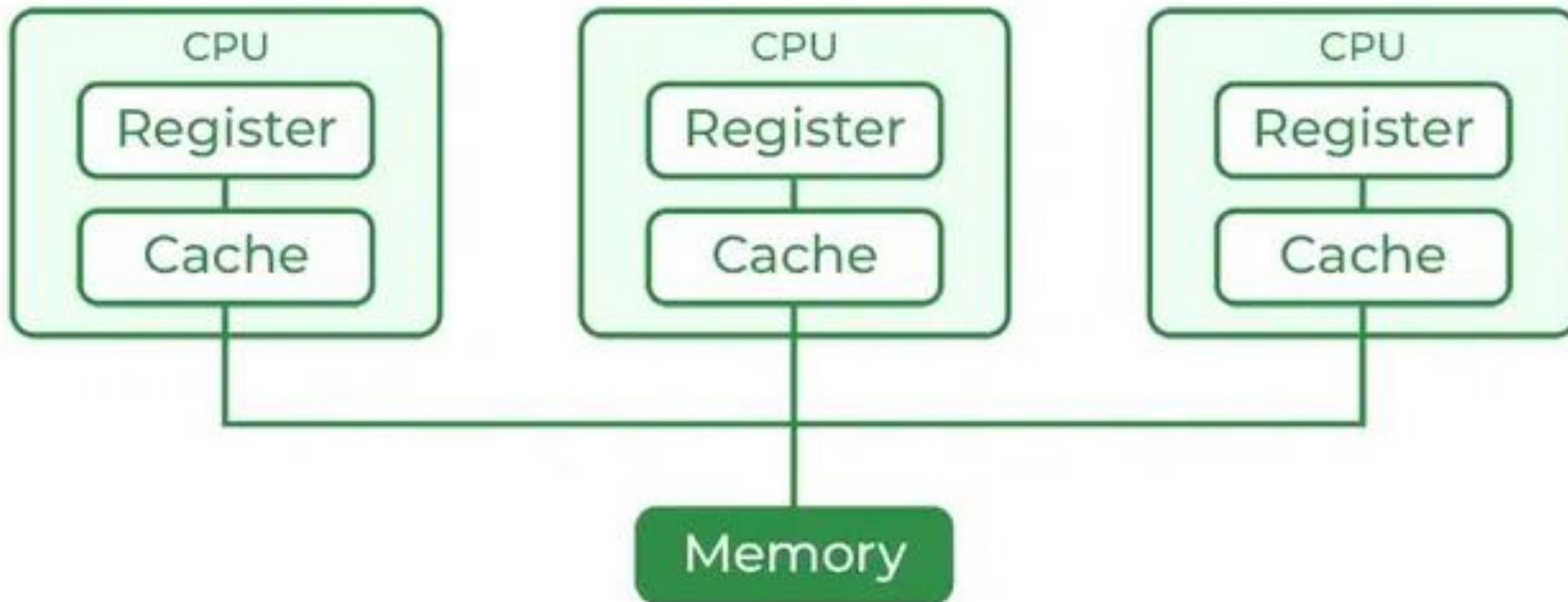
Runs multiple programs in memory at the same time to maximize CPU usage.

Multitasking



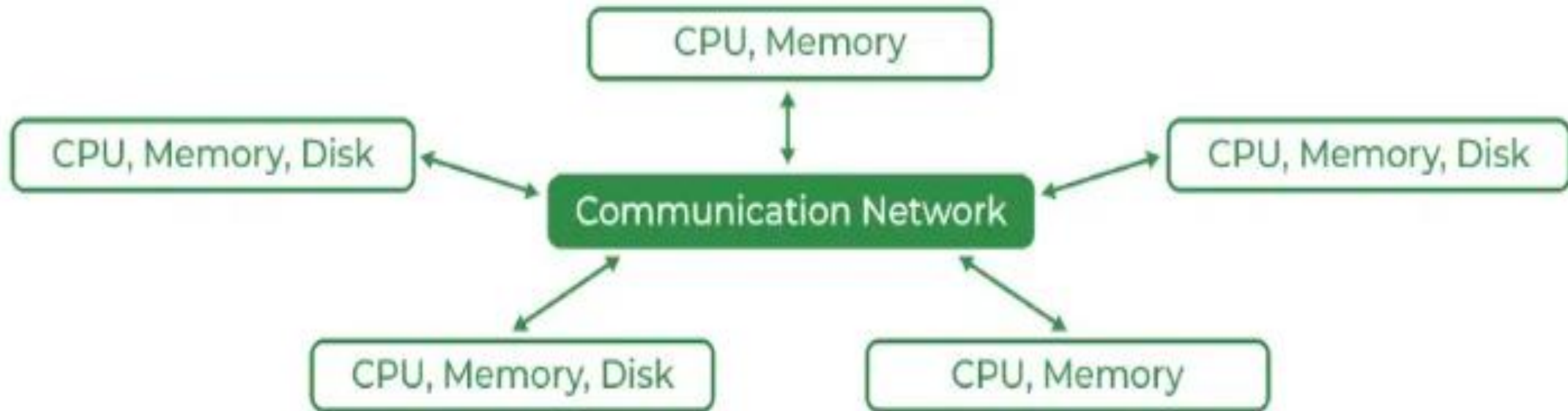
Allows multiple tasks to run by giving each a small time slice for smooth, shared system use.

Multiprocessing

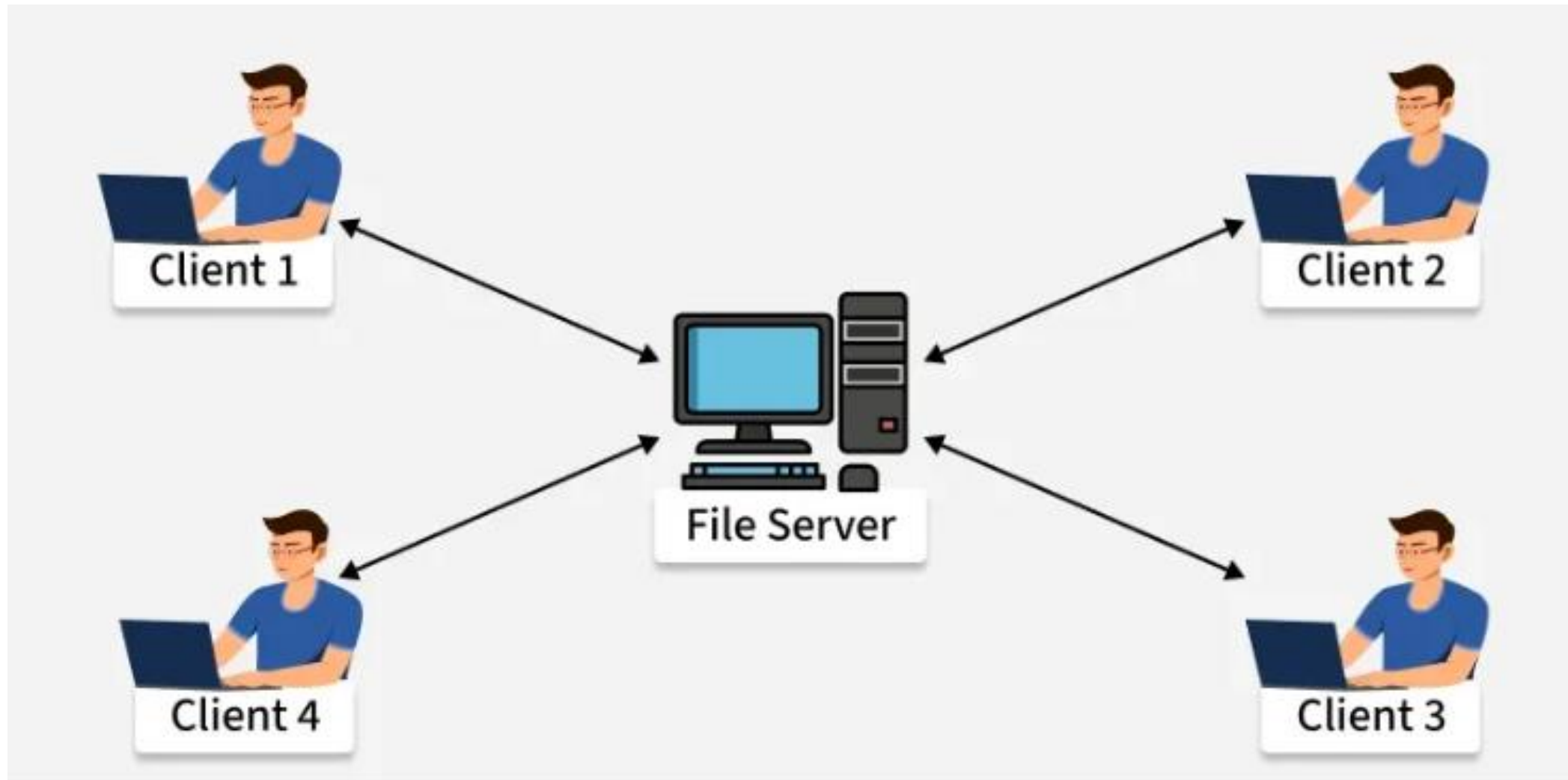


Uses two or more CPUs simultaneously to increase speed and reliability.

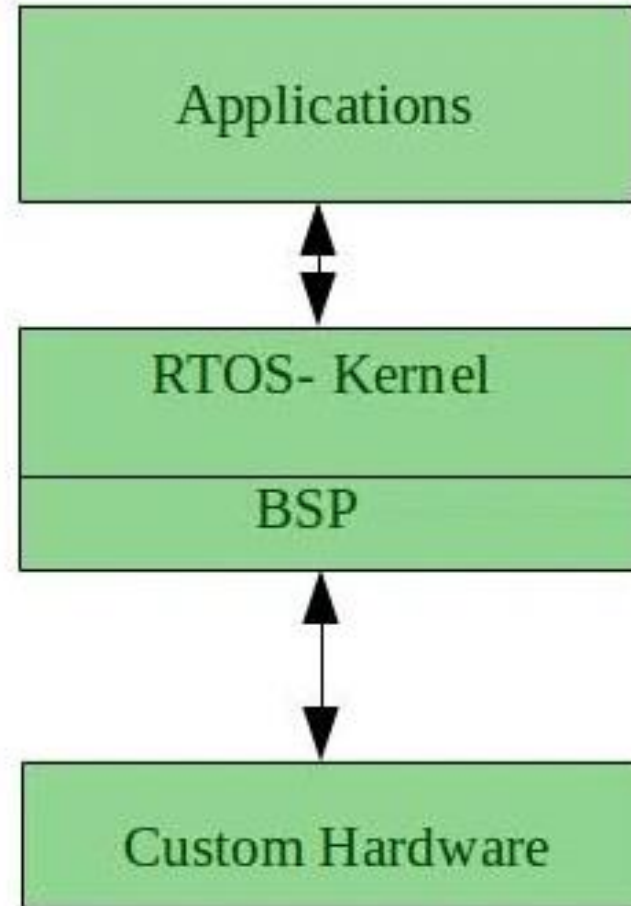
Architecture of Distributed OS



Connects multiple independent computers to function as a single coordinated system.



Manages and supports users, data, and security across connected computers within a network.



Processes data and responds immediately within strict time limits for real-time applications.

Mobile Operating System



IOS



Android



Windows Phone

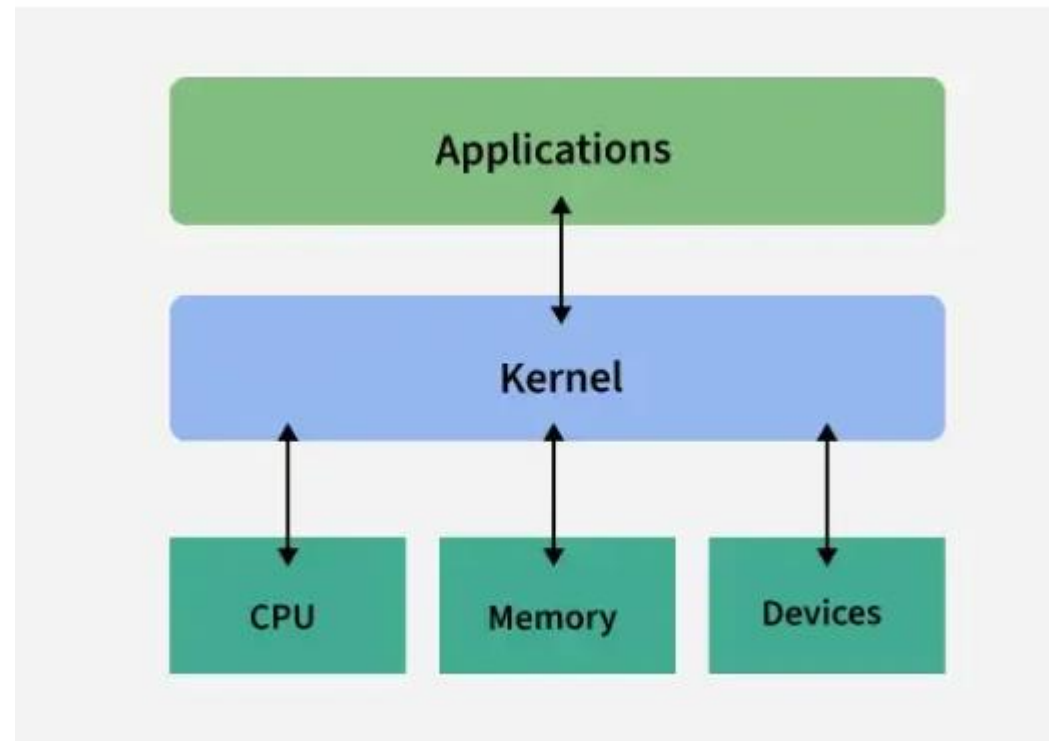


BlackBerry

Runs on smartphones and tablets, managing hardware, apps, and touch-based interfaces.

Kernel

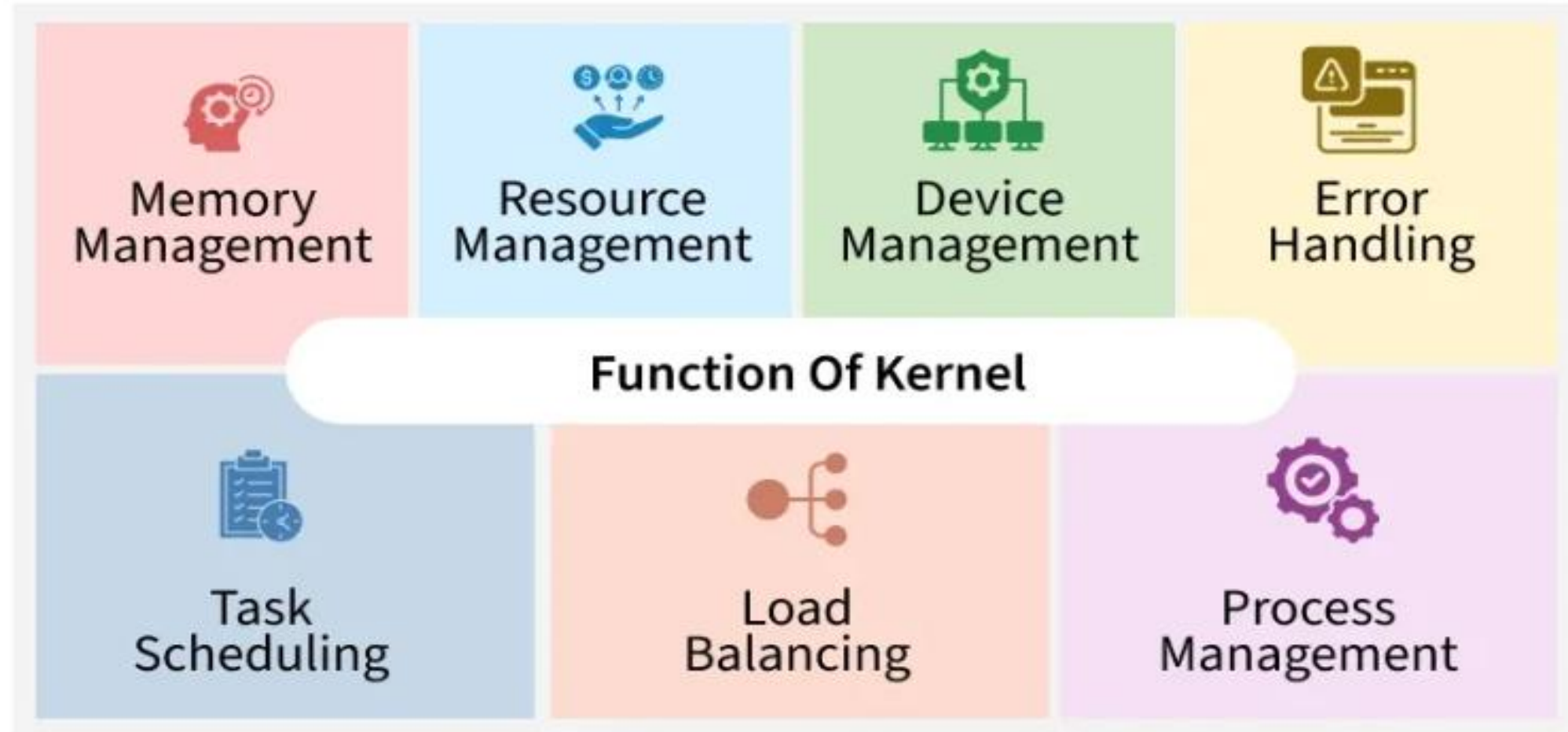
- The core component of an operating system that provides essential services.



Kernel (Operating System)

Functions of Kernel

The kernel is responsible for various critical functions that ensure the smooth operation of the computer system. These functions include:



Types of Kernels:

Monolithic Kernel

Microkernel

Hybrid Kernel

Exokernel

Nanokernel

Real-Time Kernel

Hypervisor (Virtualization Kernel)

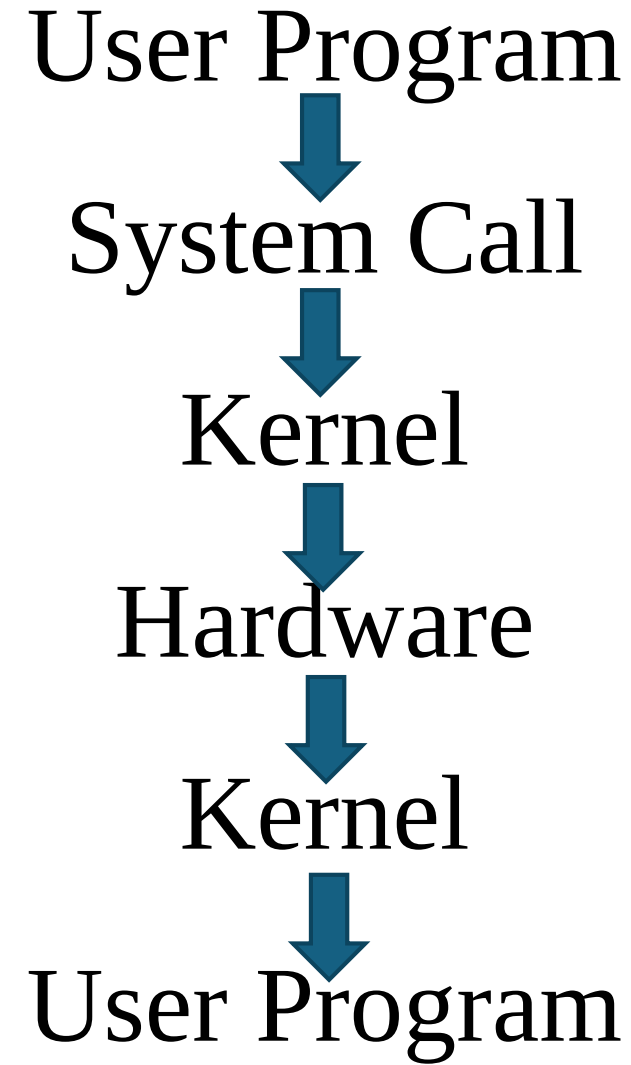
Windows NT Kernel

Linux Kernel

System Calls

- A way for programs to interact with the operating system.
- Provide the services of the operating system to the user programs.
- Only entry points into the kernel are executed in kernel mode.





Types of System Calls:

- Process Control System Calls
- File Management System Calls
- Device Management System Calls
- Information Maintenance System Calls
- Communication System Calls
- Memory Management System Calls
- Network Communication System Calls

Types Of System Calls

File System

`open()`

`close()`

`read()`

`write()`

`seek()`

Process Control

`fork()`

`exec()`

`wait()`

`exit()`

`kill()`

Memory Management

`brk()`

`sbrk()`

`mmap()`

`munmap()`

`mlock()`

`munlock()`

Interprocess Communication

`pipe()`

`socket()`

`shmget()`

`semget()`

`msgget()`

Device Management

`setConsoleMode()`

`WriteConsole()`

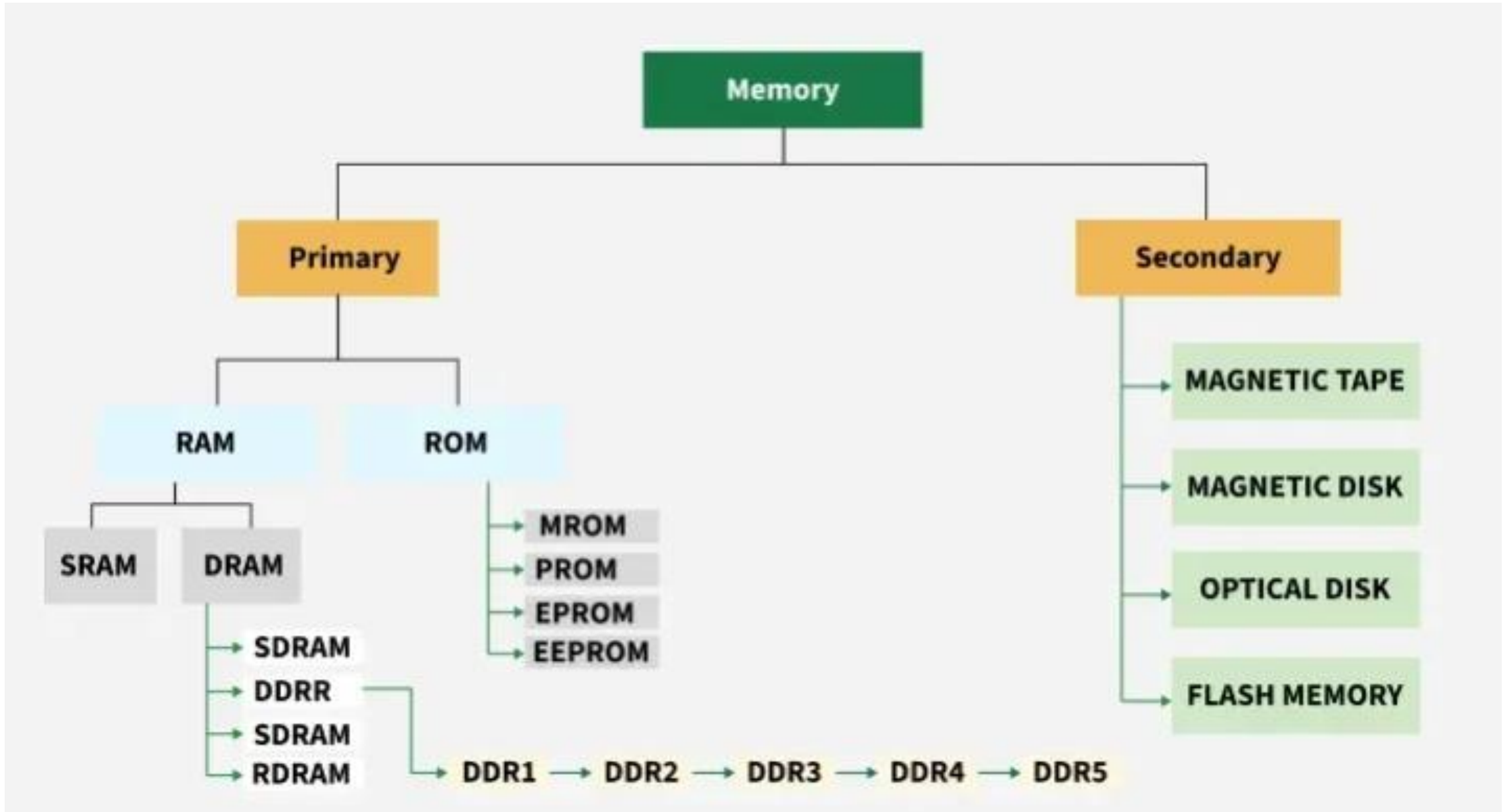
`ReadConsole()`

`open()`

`close()`

Types of Memory

- Primary Memory (Main Memory)
 - RAM
 - ROM
- Secondary Memory (Storage Devices)
 - Hard Disk Drives (HDD)
 - Solid State Drives (SSD)
 - Flash Drives (USB Drives)
 - Optical Drives (CDs, DVDs, Blu-rays)
- Cache Memory L1, L2, and L3 Cache
- Registers
- Virtual Memory
- Graphics Memory (GPU Memory)
- Memory Cards

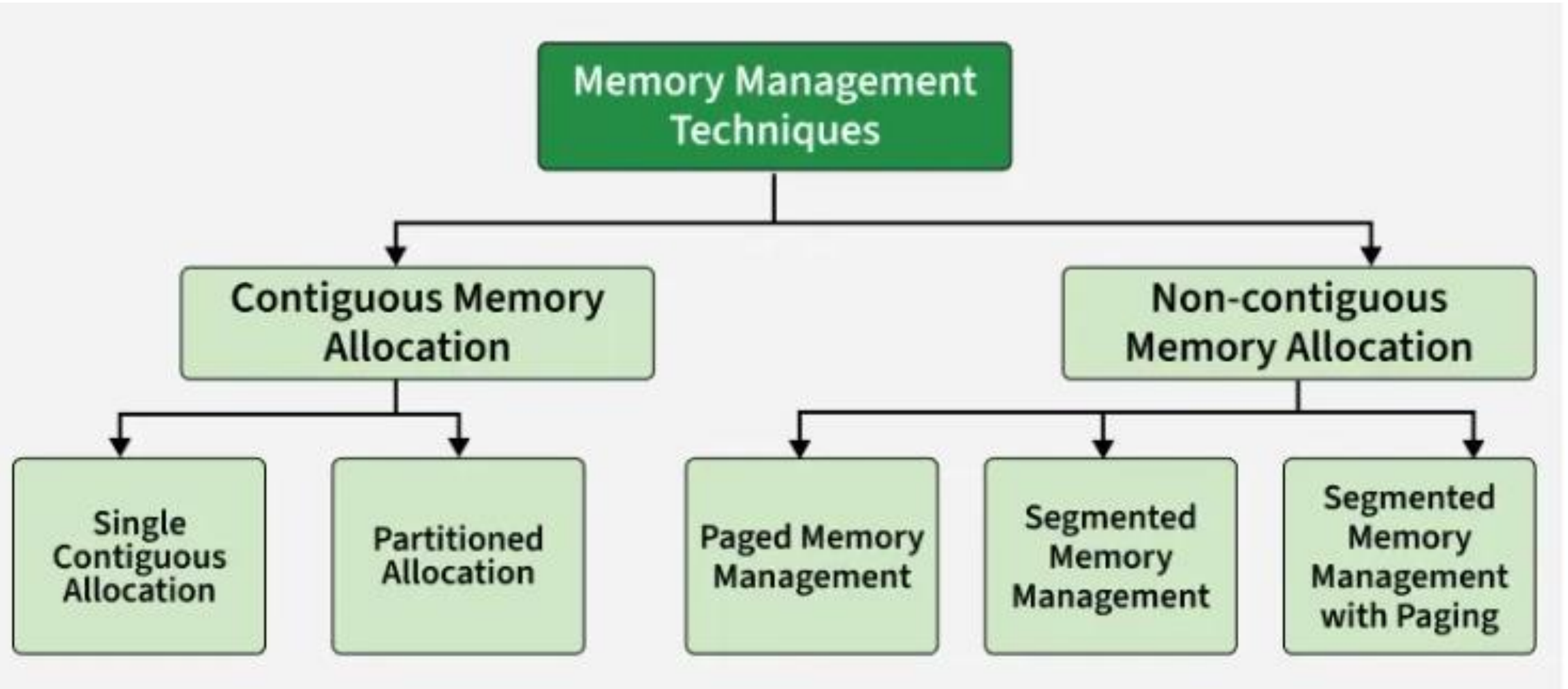


Memory Hierarchy Design



Memory Management Schemes

- Fixed Partitioning
- Variable Partitioning
- Memory Allocation Algorithms:
 - Paging
 - Demand Paging
 - Segmentation



RAID (Redundant Array of Independent Disks)

- RAID is a technology used to improve the performance and reliability of computer storage.
- RAID Levels:
 - RAID 0 (Striping)
 - RAID1(Mirroring)
 - RAID-3 (Byte-Level Striping with Dedicated Parity)
 - RAID-4 (Block-Level Striping with Dedicated Parity)
 - RAID5(Striping with Distributed Parity)
 - RAID6(Striping with Dual Parity)
 - RAID10(Striping and Mirroring)

RAID 0 (Striping)

RAID 0

Disk 0	Disk 1	Disk 2	Disk 3
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

RAID1(Mirroring)

RAID 1

Disk 0	Disk 1	Disk 2	Disk 3
0	0	1	1
2	2	3	3
4	4	5	5
6	6	7	7

RAID-3 (Byte-Level Stripping with Dedicated Parity)

RAID 3

Disk 0	Disk 1	Disk 2	Disk 3
15	16	17	P(15, 16, 17)
18	19	20	P(18, 19, 20)
21	22	23	P(21, 22, 23)
24	25	26	P(24, 25, 26)

RAID5(Striping with Distributed Parity)

RAID 5

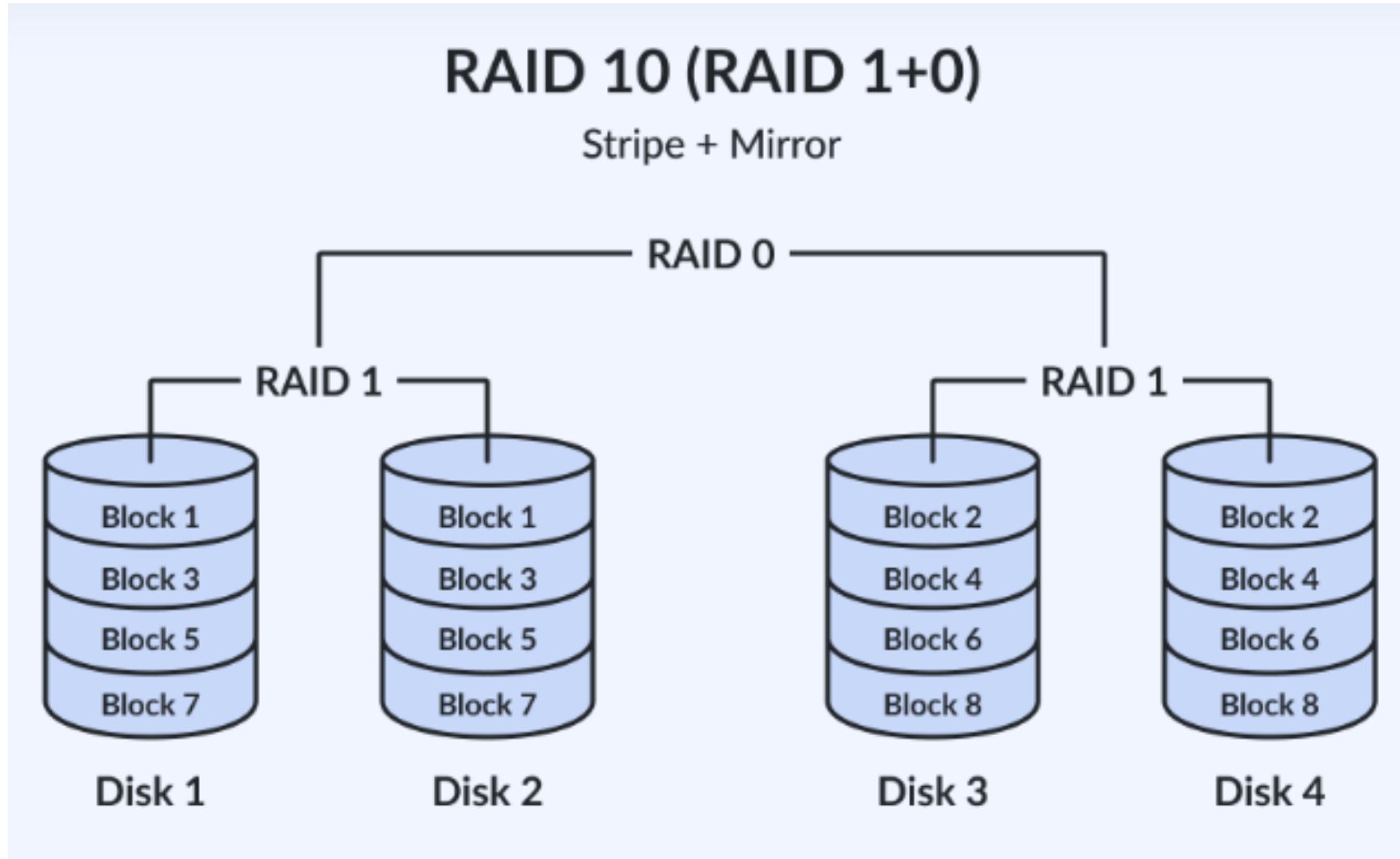
Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	1	2	3	P0
5	6	7	P1	4
10	11	P2	8	9
15	P3	12	13	14
P4	16	17	18	19

RAID6 (Striping with Dual Parity)

RAID 6

Disk 1	Disk 2	Disk 3	Disk 4
A1	B1	P(B1)	P(B1)
A2	P(B2)	P(B2)	B2
P(B3)	P(B3)	A3	B3
P(B4)	A4	A4	P(B4)

RAID10(Striping and Mirroring)

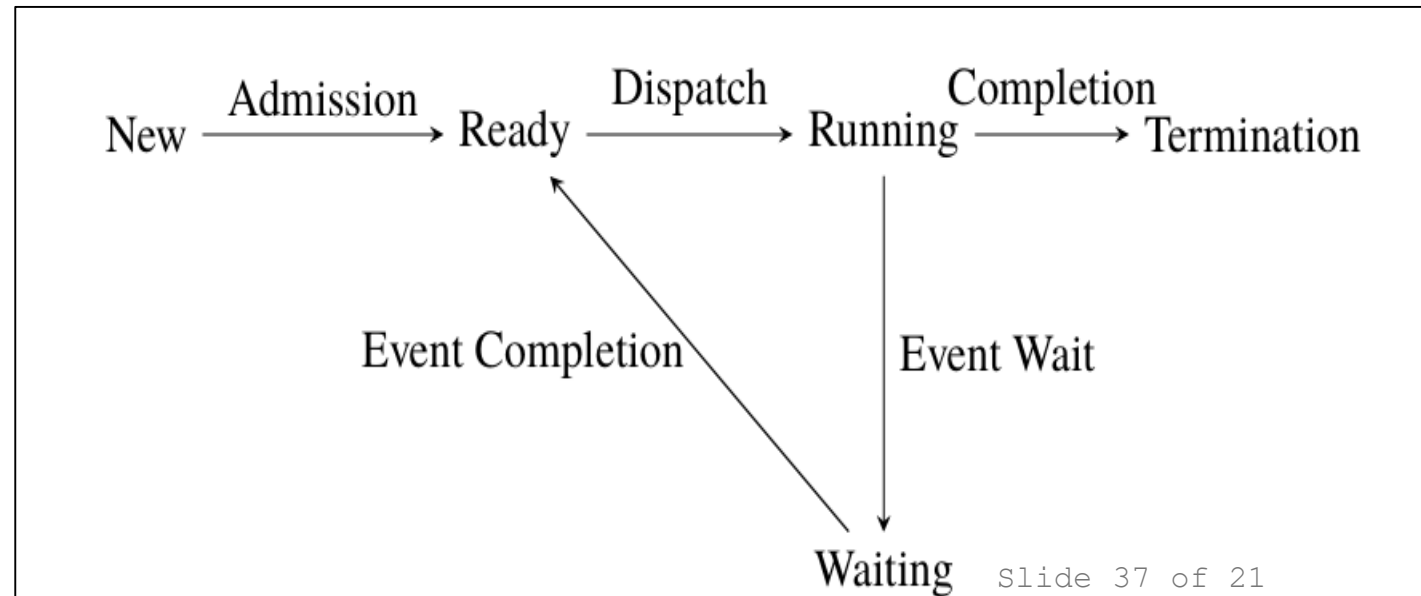


Process

- A program in execution.
- Types of Processes:
 - Foreground Process
 - Background Process
 - Parent Process:
 - Child Process
 - Daemon Process
 - Orphan Process
 - Zombie Process
- Batch Process
- User-Level Process
- System-Level Process
- Cooperative Process

Process State Transition Diagram

- New
- Ready
- Running
- Waiting (Blocked)
- Terminated



Process or Thread Synchronization

- It is employed to control the access to shared resources by multiple processes or threads.
- Some key concepts related to synchronization:
 - Critical Section
 - Mutual Exclusion
 - Semaphore
 - Mutex (Mutual Exclusion)
 - Condition Variable
- Deadlock
- Race Condition
- Atomic Operations
- Barrier
- Read-Write Lock
- Busy Waiting

Process Synchronization in Operating System

✦ Definition

Process Synchronization is a mechanism in an operating system that ensures **multiple processes or threads execute in a controlled manner when sharing common resources**, so that **data consistency and correctness** are maintained.

When processes execute concurrently, improper synchronization may lead to **race conditions, inconsistent data, or system failure**.

Why Process Synchronization is Needed

When two or more processes:

- Share **common data**
- Access **critical resources simultaneously**
- Execute **concurrently**

→ Synchronization is required to **control access** to shared resources.

Problems Without Synchronization

1.Race Condition

Final output depends on the execution order of processes.

2.Data Inconsistency

Shared data may get corrupted.

3.Deadlock

Processes wait forever for resources.

4.Starvation

A process never gets CPU or resources.

Critical Section Problem

A **critical section** is a part of the program where **shared variables or resources** are accessed.

Requirements of Critical Section:

1. Mutual Exclusion

Only one process can enter the critical section at a time.

2. Progress

Selection of the next process must not be postponed indefinitely.

3. Bounded Waiting

A limit exists on the number of times other processes can enter the CS before a waiting process.

Advantages

- ✓ Prevents data inconsistency
- ✓ Maintains system stability
- ✓ Enables safe concurrency

Disadvantages

- ✗ Overhead
- ✗ Deadlock & starvation
- ✗ Complexity in design

Cooperative Process and Independent Process

Cooperative Process

A **cooperative process** is a process that **shares data or resources** with other processes.

Its execution **affects and is affected by** other processes.

Characteristics

- Shared memory or shared files
- Requires IPC mechanisms
- Synchronization is essential
- Risk of race conditions

Examples (Any 3)

1.Producer–Consumer Processes

Producer adds data to buffer; consumer removes data.

2.Multiple Processes Updating a Database

Each process accesses common records.

3.Web Server Handling Multiple Clients

Processes share logs, memory, or sockets.

Independent Process

An **independent process** is a process that **does not share data or resources** with any other process. Its execution **does not affect** and is **not affected by** other processes.

Characteristics

- No shared variables or files
- No inter-process communication (IPC)
- Execution is completely isolated
- No synchronization required

Examples (Any 3)

1. Calculator Application

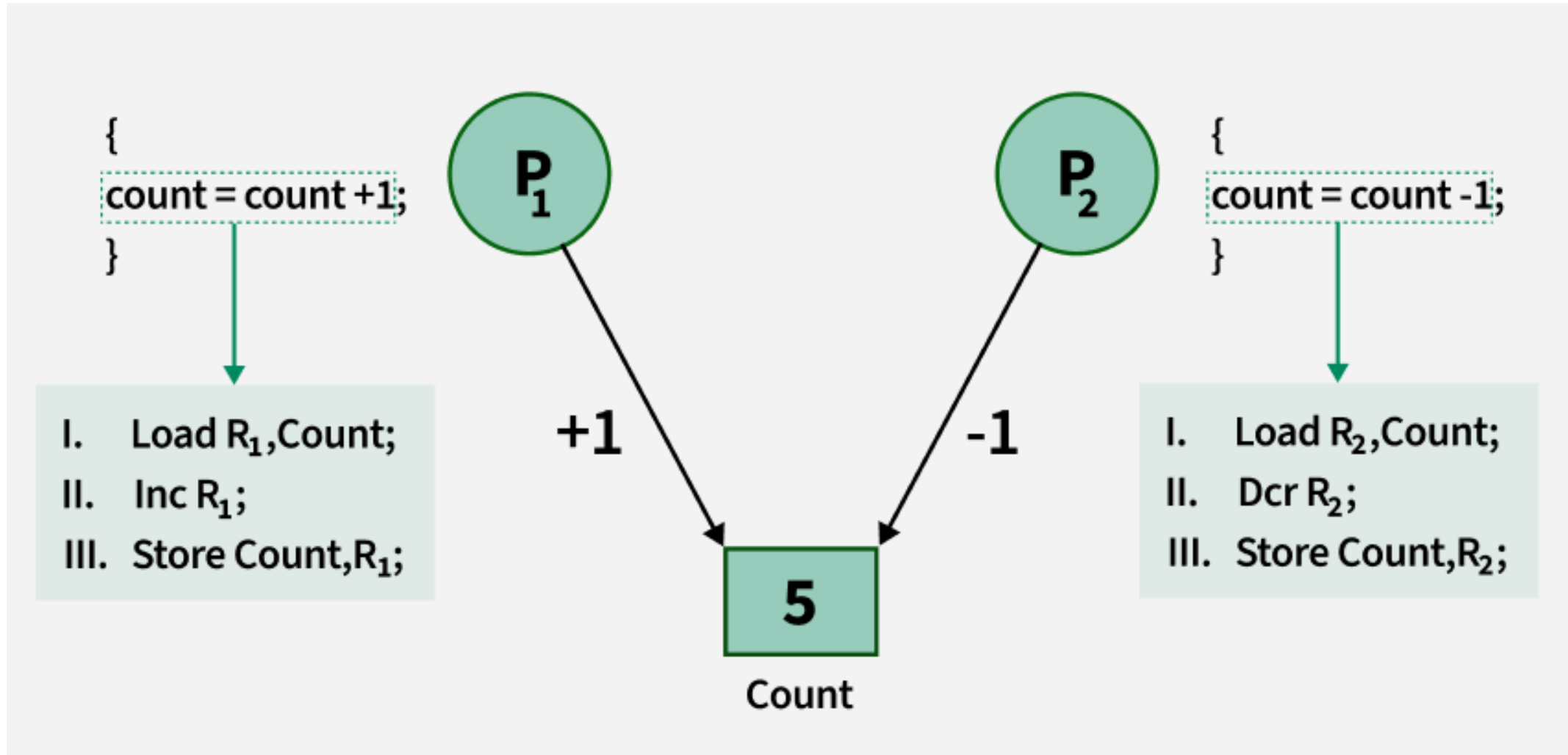
Performing calculations without interacting with other programs.

2. Text Editor Editing a Local File

Editing a file that is not shared with other processes.

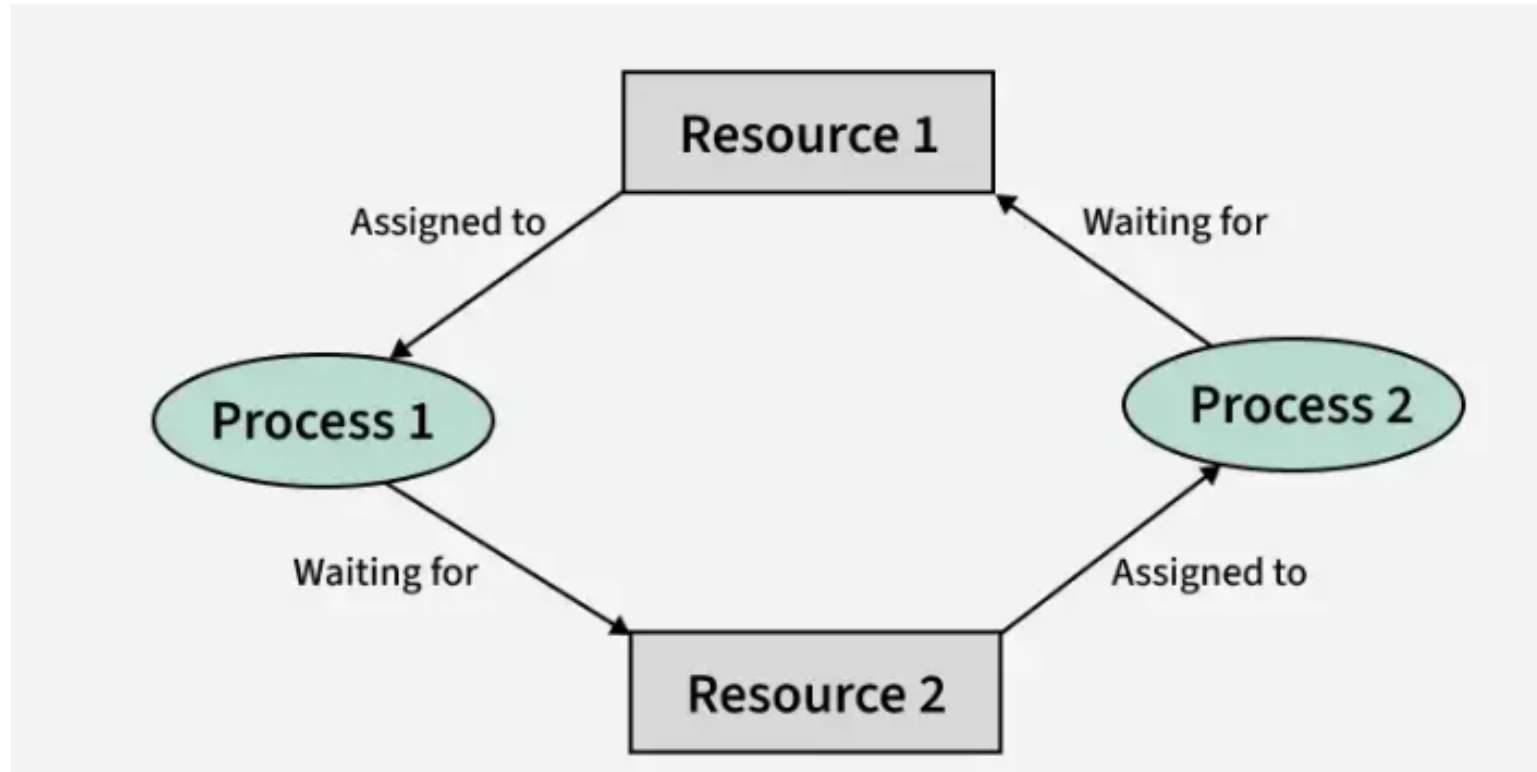
3. Offline Music Player

Playing stored music without accessing shared resources.



Deadlock

- A deadlock is a situation in computing where two or more processes are unable to proceed because each is waiting for the other to release a resource.



Technical Examples

1. Printer–Scanner Deadlock

1. Process P1 holds the **printer** and waits for the **scanner**.
2. Process P2 holds the **scanner** and waits for the **printer**.
3. Neither process can proceed → deadlock.

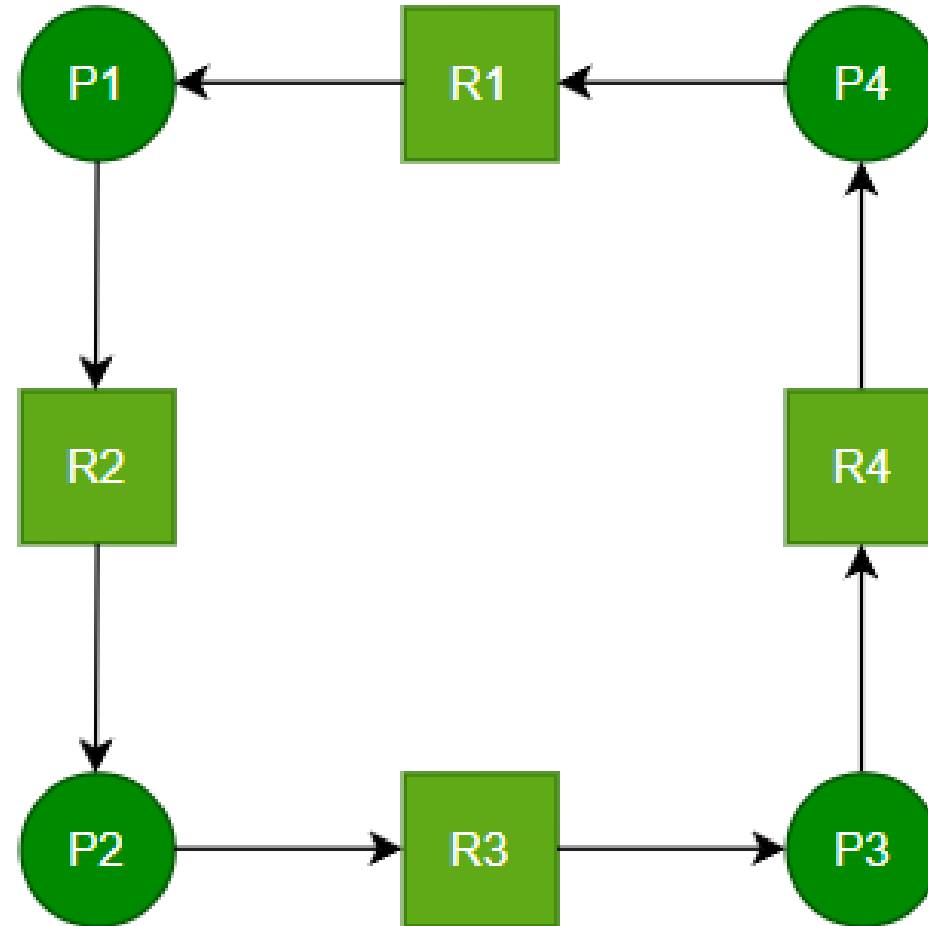
2. File Lock Deadlock in Databases

1. Transaction T1 locks **File A** and waits for **File B**.
2. Transaction T2 locks **File B** and waits for **File A**.
3. Both transactions remain blocked forever.

Necessary Conditions for Deadlock in OS

Deadlock can arise if the following four conditions hold simultaneously (Necessary Conditions)

- 1.Mutual Exclusion:** Only one process can use a resource at any given time i.e. the resources are non-sharable.
- 2.Hold and Wait:** A process is holding at least one resource at a time and is waiting to acquire other resources held by some other process.
- 3.No Preemption:** A resource cannot be taken from a process unless the process releases the resource.
- 4.Circular Wait:** set of processes are waiting for each other in a circular fashion. For example, imagine four processes **P1**, **P2**, **P3**, and **P4** and four resources **R1**, **R2**, **R3**, and **R4**



The above image demonstrates a circular wait deadlock.

One Combined Real-Life Example (Easy to Remember)

Two people + one pen + one paper

- Pen and paper can be used by only one person (**Mutual Exclusion**)
- Each person holds one item and waits for the other (**Hold and Wait**)
- Items cannot be taken forcibly (**No Preemption**)
- A waits for B, B waits for A (**Circular Wait**)

Deadlock Condition	Meaning (Simple Language)	Technical Example (OS)	Real-Life Example
Mutual Exclusion	A resource can be used by only one process at a time	Only one process can use a printer; other processes must wait	Only one person can use a washroom at a time
Hold and Wait	A process holds one resource and waits for another	Process P1 holds a file and waits for a CPU, while P2 holds CPU and waits for the file	One person holds a pen and waits for paper
No Preemption	A resource cannot be taken back forcefully	OS cannot take a locked file from a process until it releases it	You cannot snatch a phone from someone while they are using it
Circular Wait	A circle of waiting is formed among processes	P1 waits for P2 → P2 waits for P3 → P3 waits for P1	Three people each waiting for the other to move first

Deadlock Avoidance Methods

Avoidance Method	Meaning (Simple Language)	Technical Example (OS)	Real-Life Example
Safe State Check	System checks in advance whether granting a request is safe	OS grants resources only if the system remains in a safe state	Giving money only if you know you can still pay all bills
Banker's Algorithm	OS acts like a banker, lending resources carefully	Process declares maximum need; resources allocated only if safe	Bank gives a loan only after checking your repayment capacity
Resource Allocation Graph (with claims)	System uses a graph to decide whether to allow a request	Request is granted only if it does not form a cycle	Allowing road entry only if traffic flow stays clear

Banker's Algorithm

Definition (Very Simple Language)

Banker's Algorithm is a **deadlock avoidance algorithm** used by the operating system.

It works like a **bank manager** who gives loans only when he is sure that **all customers** **can return the money safely**.

The OS allocates resources **only if the system remains in a safe state**, so deadlock never occurs.

Steps of Banker's Algorithm (Easy Words)

1. Know Maximum Need

Each process tells the OS the **maximum number of resources** it may need.

2. Know Allocated Resources

OS keeps track of how many resources are **already given** to each process.

3. Calculate Remaining Need

Need = Maximum – Allocated

4. Check Available Resources

OS checks how many resources are **free**.

5. Safety Check

If a process's need $\leq$ available resources,

→ allow it to finish

→ release its resources

→ update available resources.

6. Safe Sequence

If all processes can finish one by one, the system is in a **safe state**

Numerical Example of Banker's Algorithm

Given Data

- Total Resources = 10
- Processes = P0, P1, P2

Step 1: Allocation & Maximum Requirement

Process	Allocated	Maximum
P0	3	7
P1	2	5
P2	2	3

Step 2: Calculate Available Resources

Allocated total = $3 + 2 + 2 = 7$

Available = $10 - 7 = 3$

Step 3: Calculate Need

Need = Maximum – Allocated

Process	Maximum	Allocated	Need
P0	7	3	4
P1	5	2	3
P2	3	2	1

Step 4: Safety Check (Line-by-Line Explanation)

✓ Available = 3

- P0 needs 4 → ✗ cannot run (needs more than available)
- P1 needs 3 → ✓ can run
- P2 needs 1 → ✓ can run

Let's choose P2 first.

Step 5: Execute P2

- P2 finishes and releases its allocated resources = 2
- New Available = $3 + 2 = 5$

Step 6: Repeat Safety Check

✓ Available = 5

- P0 needs 4 → ✓ can run
- P1 needs 3 → ✓ can run

Choose P1 next.

Step 7: Execute P1

- P1 releases its resources = 2
 - New Available = $5 + 2 = 7$
-

Step 8: Execute P0

- P0 needs 4 → ✓ available is 7
- P0 finishes and releases 3
- Final Available = 10

Safe Sequence

$P2 \rightarrow P1 \rightarrow P0$

★ Final Conclusion (Exam Line)

- System is in a **SAFE STATE**
- Deadlock will not occur
- Banker's Algorithm allows allocation only when a **safe sequence exists**

Numerical Example 2

Given

- Total Resources = 12
- Processes = P0, P1, P2

Step 1: Allocation & Maximum

Process	Allocated	Maximum
P0	2	6
P1	3	5
P2	2	4

12 **34** Numerical Example 3 (Moderate Level – Exam Favorite)

Given

- Total Resources = 15
- Processes = P0, P1, P2, P3

Step 1: Allocation & Maximum

Process	Allocated	Maximum
P0	1	4
P1	3	6
P2	4	7
P3	2	5

5-Process Banker's Algorithm Numerical

Given

- Total Resources = 10
- Processes = P0, P1, P2, P3, P4

Step 1: Allocation and Maximum Requirement

Process	Allocated	Maximum
P0	1	3
P1	2	4
P2	3	6
P3	2	5
P4	1	2

5-Process UNSAFE State Example

Given

- Total Resources = 10
- Processes = P0, P1, P2, P3, P4

Step 1: Allocation & Maximum Requirement

Process	Allocated	Maximum
P0	1	3
P1	2	4
P2	3	6
P3	2	5
P4	2	3

Resource Allocation Graph (RAG)

A Resource Allocation Graph (RAG) is a directed graph used in Operating Systems to represent the allocation of resources to processes and to detect or prevent deadlocks.

- Nodes: Two types
 1. Process Nodes ($P_0, P_1, \dots$) → represent processes
 2. Resource Nodes ($R_0, R_1, \dots$) → represent resources
- Edges: Two types
 1. Request Edge ($P \rightarrow R$): Process is requesting a resource
 2. Assignment Edge ($R \rightarrow P$): Resource is allocated to process

Example 1: Simple Deadlock Scenario

Scenario:

- Processes: P0, P1
- Resources: R0, R1 (1 instance each)
- Situation:
 - P0 holds R0, requests R1
 - P1 holds R1, requests R0

Step 1: Draw Nodes

- Circles → P0, P1
- Squares → R0, R1

Step 2: Draw Edges

- Assignment edges (Resource $\rightarrow$ Process):
 - $R0 \rightarrow P0$ ($R0$ allocated to $P0$)
 - $R1 \rightarrow P1$ ($R1$ allocated to $P1$)
- Request edges (Process $\rightarrow$ Resource):
 - $P0 \rightarrow R1$
 - $P1 \rightarrow R0$

Step 3: Analyze

- A cycle exists: $P0 \rightarrow R1 \rightarrow P1 \rightarrow R0 \rightarrow P0$
- Conclusion: Deadlock occurs

Example 2: No Deadlock (Safe State)

Scenario:

- **Processes:** P0, P1
- **Resources:** R0, R1 (1 instance each)
- **Situation:**
 - P0 holds R0
 - P1 requests R0

Example 3: Multiple Instances of Resources

- **Processes:** P0, P1
- **Resource R0 has 2 instances**
- **Situation:**
 - P0 holds 1 instance of R0, requests 1 more
 - P1 holds 1 instance of R0

CPU Scheduling

- CPU scheduling manages the execution of processes in a computer system with a single central processing unit (CPU)
- A **CPU Scheduling Algorithm** is a method used by the **Operating System** to decide **which process gets the CPU, when, and for how long**, when multiple processes are present in the **ready queue**.

Objectives of Scheduling

Maximize CPU utilization

Minimize waiting time

Minimize turnaround time

Minimize response time

Ensure fairness

Avoid Deadlock

Scheduling Criteria (Formulas)

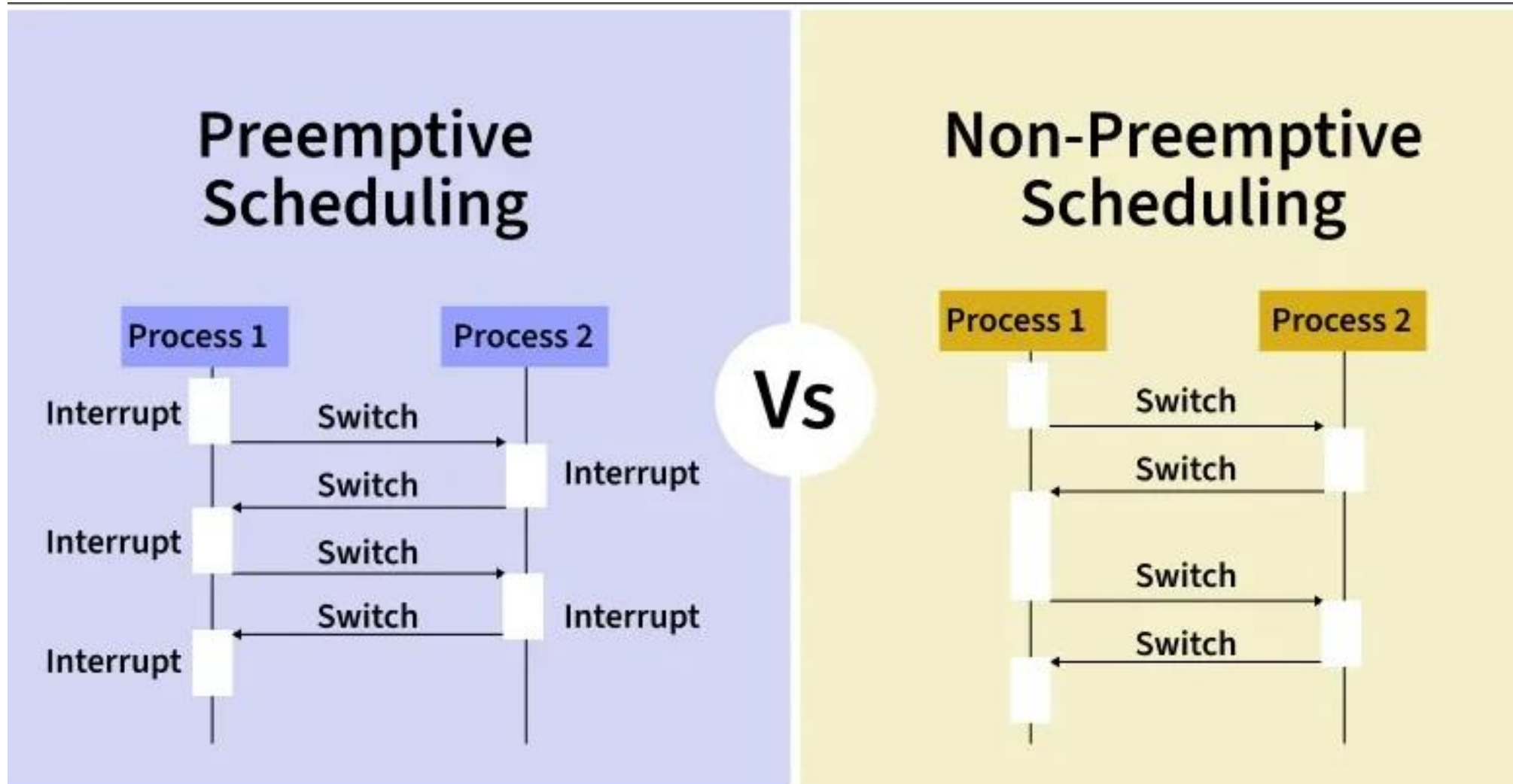
Let

- **AT** = Arrival Time
- **BT** = Burst Time
- **CT** = Completion Time

Formulas

- **Turnaround Time (TAT)** = $CT - AT$
- **Waiting Time (WT)** = $TAT - BT$
- **Average WT** = Sum of WT / Number of processes
- **Average TAT** = Sum of TAT / Number of processes

Preemptive and Non-Preemptive Scheduling



Preemptive Scheduling	Non-Preemptive Scheduling
In this resources(CPU Cycle) are allocated to a process for a limited time.	Once resources(CPU Cycle) are allocated to a process, the process holds it till it completes its burst time or switches to waiting state
Process can be interrupted in between.	Process can not be interrupted until it terminates itself or its time is up
If a process having high priority frequently arrives in the ready queue, a low priority process may starve	If a process with a long burst time is running CPU, then later coming process with less CPU burst time may starve
Examples of preemptive scheduling are Round Robin and Shortest Remaining Time First	Examples of non-preemptive scheduling are First Come First Serve and Shortest Job First

First Come First Serve (FCFS)

(Non-Preemptive)

Definition

The process that **arrives first** gets the CPU first.

Characteristics

- Simple
- FIFO order
- Non-preemptive

Advantages

- Easy to implement

Disadvantages

- Poor average waiting time



L P U

NAAC
GRADE **A++**

Shortest Job First (SJF)

(Non-Preemptive)

Definition

The process with the **smallest burst time** executes first.

Characteristics

- Optimal scheduling
- Requires burst time knowledge

Advantages

- Minimum average waiting time

Disadvantages

- Starvation possible
- Burst time prediction difficult

Shortest Remaining Time First (SRTF)

(Preemptive SJF)

Definition

CPU is allocated to the process with **shortest remaining burst time**.

Advantages

- Best average response time

Disadvantages

- Starvation
- High context switching

Priority Scheduling

Definition

CPU is allocated based on **priority number**.

Types

- Preemptive
- Non-Preemptive

Advantages

- Important tasks execute first

Disadvantages

- Starvation
- Priority inversion

Round Robin (RR)

(Preemptive)

Definition

Each process gets CPU for a **fixed time quantum (TQ)**.

Characteristics

- Time-sharing
- Preemptive

Advantages

- Fair scheduling
- No starvation

Disadvantages

- Large TQ $\rightarrow$ FCFS
- Small TQ $\rightarrow$ High overhead

Comparison Table

Algorithm	Preemptive	Starvation	Response
FCFS	No	No	Poor
SJF	No	Yes	Good
SRTF	Yes	Yes	Best
Priority	Both	Yes	Depends
Round Robin	Yes	No	Good

CPU Scheduling Terms

1. **Arrival Time (AT):** Arrival Time (AT) is the time at which a process enters the ready queue and becomes ready for execution.
2. **Burst Time (BT):** Burst Time (BT) is the total CPU time required by a process to complete its execution.
3. **Completion Time (CT) :** Completion Time (CT) is the time at which a process finishes its execution completely.
4. **Turnaround Time (TAT / TT) :** Turnaround Time (TAT) is the total time taken from arrival of the process to its completion.
5. **Waiting Time (WT) :** Waiting Time (WT) is the total time a process spends waiting in the ready queue, excluding execution time.
6. **Response Time (RT) :** Response Time (RT) is the time between arrival of a process and the first time it gets CPU

Term	Meaning (Simple)	Student Example
AT	When process arrives	Student enters exam hall
BT	CPU time needed	Time to write exam
CT	When process finishes	Time of submission
TAT	Total time in system	Entry → Exit
WT	Time spent waiting	Waiting before writing
RT	Time till first CPU	Time till question paper

CPU Scheduling – Complete Formula List

Turnaround Time (TAT / TT)

$$\text{TAT} = \text{CT} - \text{AT}$$

Waiting Time (WT)

$$\text{WT} = \text{TAT} - \text{BT}$$

OR

$$\text{WT} = \text{CT} - \text{AT} - \text{BT}$$

Response Time (RT)

$$\text{RT} = \text{First CPU Allocation Time} - \text{AT}$$

Average Waiting Time

$$Avg\ WT = \frac{\sum WT}{n}$$

Average Turnaround Time

$$Avg\ TAT = \frac{\sum TAT}{n}$$

Average Response Time

$$Avg\ RT = \frac{\sum RT}{n}$$

Interprocess Communication (IPC) in Operating System

Interprocess Communication (IPC) is a mechanism provided by the **Operating System** that allows **processes to communicate with each other and synchronize their actions.**

Since processes execute in separate address spaces, IPC is required to exchange data and coordinate execution.

Need for IPC

IPC is required when:

- Processes need to **share data**
- Processes must **coordinate** or **synchronize** their execution
- Tasks are divided into **multiple cooperating processes**
- Efficient utilization of **CPU and system resources**

Example:

- A web browser (one process) downloading data and another process displaying it

Types of Processes (Based on IPC)

1.Independent Processes

1. Do not share data with other processes
2. Execution is not affected by other processes
3. Example: Printing a document, playing a local video

2.Cooperating Processes

1. Share data and affect each other's execution
2. Require IPC mechanisms
3. Example: Producer–Consumer problem

A. Shared Memory Model

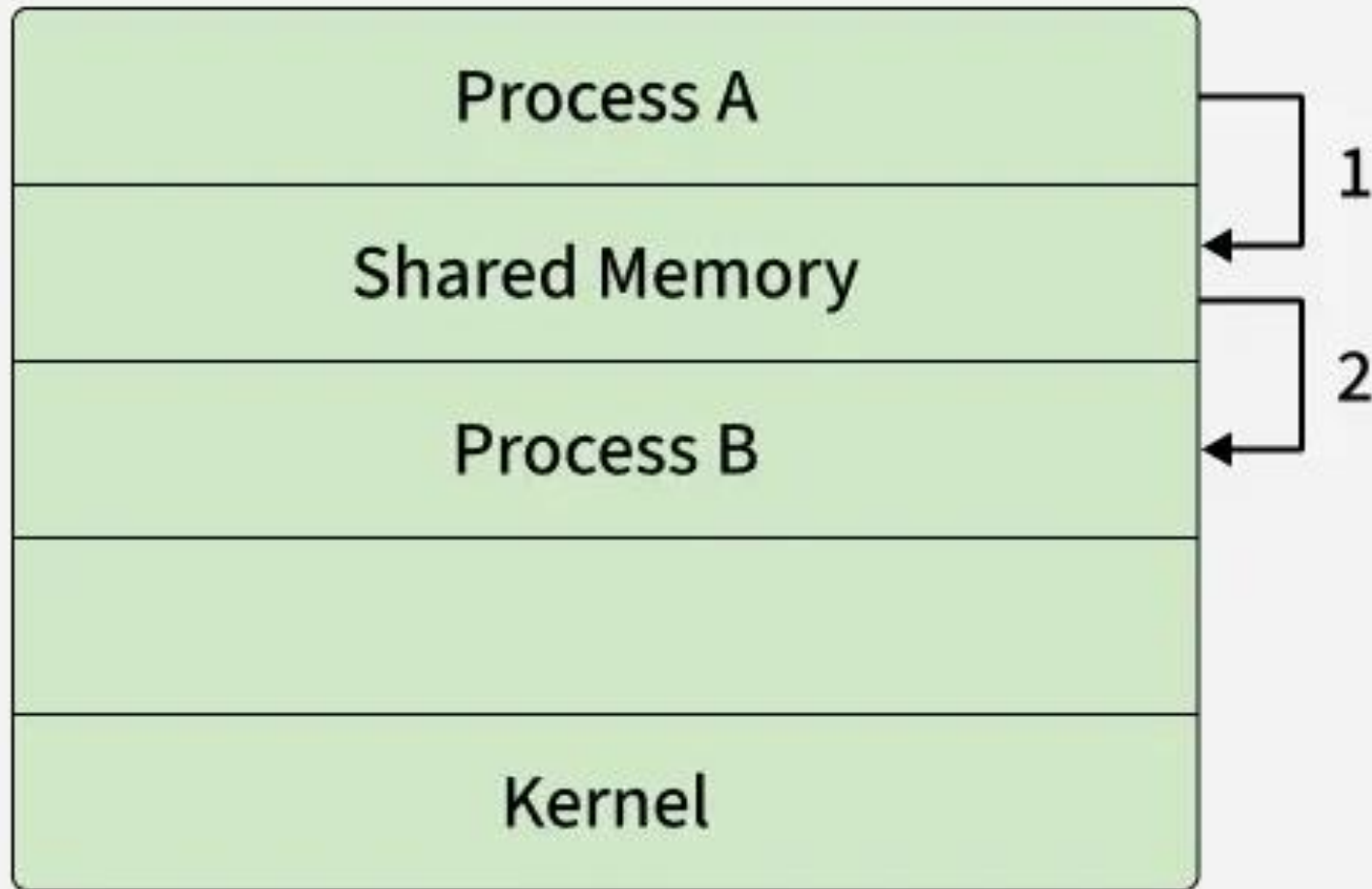
1. Concept

- Processes share a **common memory region**
- OS creates the shared memory, but processes manage communication
- Faster than message passing (no kernel involvement after setup)

2. Working

- OS creates a shared memory segment
- Processes attach to it
- Data is read/written directly
- Synchronization is required.

Shared Memory



Example

Producer–Consumer Problem

- Producer writes data into shared buffer
- Consumer reads data from same buffer

4. Advantages

- Very fast
- Efficient for large data transfer

5. Disadvantages

- Complex to implement
- Synchronization issues (race condition, deadlock)

Message Passing Model

1. Concept

- Processes communicate by **sending and receiving messages**
- No shared address space
- Suitable for distributed systems

2. Basic Operations

- `send(message)`
- `receive(message)`

3. Types of Communication

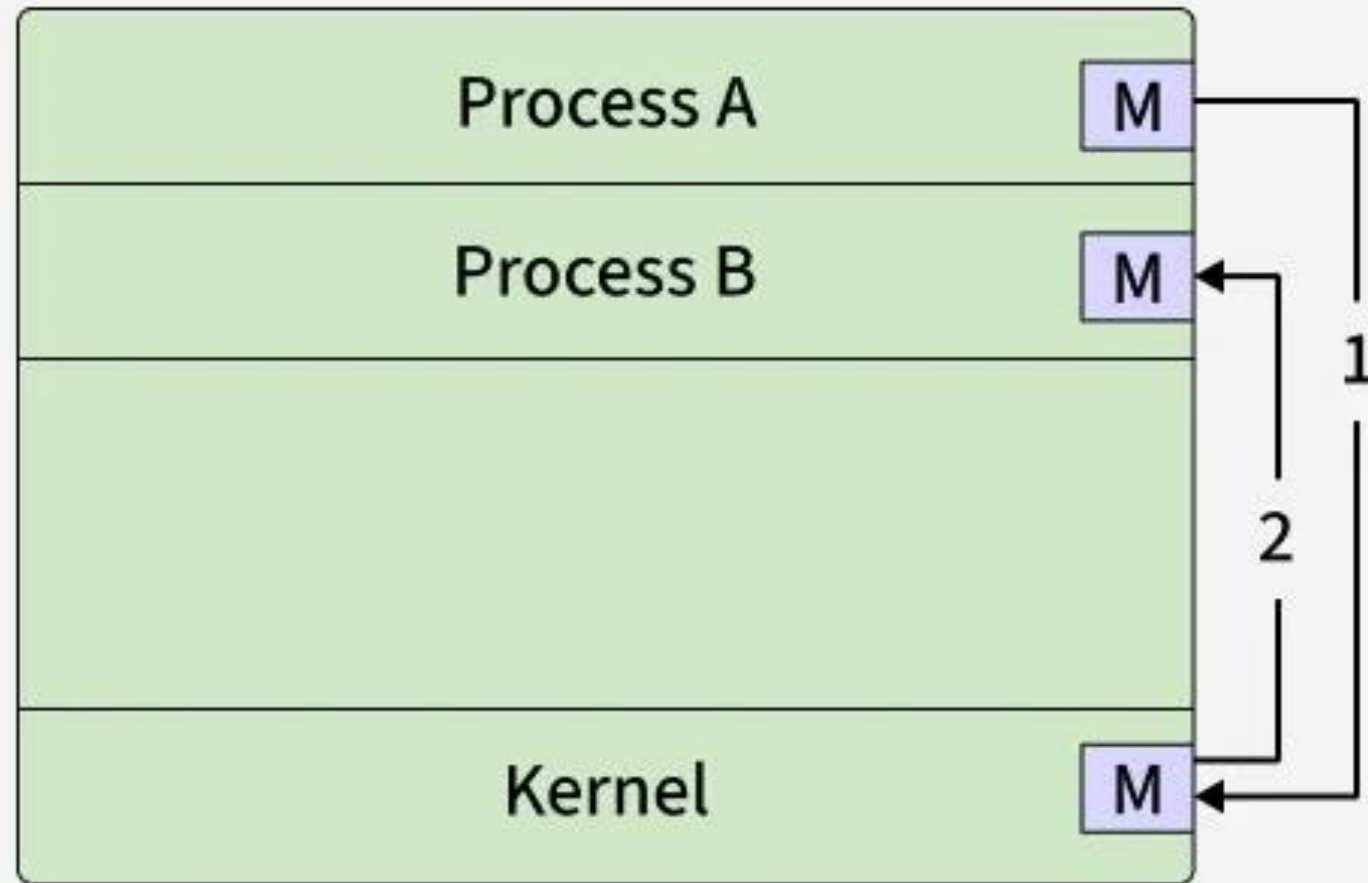
- **Direct Communication**

- Processes explicitly name each other
- Example: `send(P1, message)`

- **Indirect Communication**

- Messages are sent via **mailboxes / ports**
- Multiple processes can communicate

Message Passing



5. Advantages

- Easy to implement
- No synchronization problems like shared memory

6. Disadvantages

- Slower than shared memory
- Kernel overhead

Web Links For Reference (Complete Video Course)

- <https://youtube.com/playlist?list=PLsylUObW5M3CAGT6OdubyH6FztKfJCcFB&si=v1v4jchTBSJYXOwq>
- <https://youtube.com/playlist?list=PLyqSpQzTE6M9SYI5RqwFYtFYab94gJpWk&si=ilqHadOgkv5osVpj>
- https://youtube.com/playlist?list=PLBlnK6fEyqRiVhbXDGLXDk_OQAeuVcp2O&si=7ELRRSTFhYXv5-xg

Web Links For Reference (Study Material)

- <https://www.geeksforgeeks.org/operating-systems/>
- <https://www.javatpoint.com/operating-system>
- https://www.tutorialspoint.com/operating_system/os_overview.htm

NEXT----> Computer Networks

- Foundations of Computer Networks
- Network Topologies
- Transmission Medium
- Understanding OSI and TCP/IP Models
- Subnetting and Routing
- Key Protocols: HTTP, SMTP, POP/IMAP, FTP, DNS, DHCP,